

Citation: 3

Lossless Compression of Large Language Model-Generated Text via Next-Token Prediction

Yu Mao
MBZUAI
Abu Dhabi
yu.mao@mbzuai.ac.ae

Holger Pirk
Imperial College London
England
hlgr@ic.ac.uk

Chun Jason Xue
MBZUAI
Abu Dhabi
jason.xue@mbzuai.ac.ae

Abstract

As the deployment and utilization of LLMs continue to expand, the volume of LLM-generated data is increasing at an unprecedented rate. Therefore, the need for *effective and lossless compression of LLM-generated data has become increasingly critical in text data management*. However, compressing LLM-generated data poses significant challenges compared to traditional text data, such as human-generated or machine-generated content. Traditional machine-generated data is predominantly derived from computational processes or device outputs, characterized by its highly structured and discrete nature, often limited to low-level information such as labels or numerical values. As a result, conventional lossless compression algorithms are able to exploit this structure to efficiently compress such data. In contrast, the complexity and diversity of LLM-generated data require different approaches to achieve effective lossless compression. To this end, this work conducts the first investigation into effective lossless compression methods for LLM-generated data. Interestingly, since LLMs are trained through next-token prediction, we observe that LLM-generated data exhibits a high predictability for LLMs, which allows LLMs to achieve remarkable lossless compression of LLM-generated data. We conduct extensive experiments with 14 representative LLMs and 8 LLM-generated datasets from different categories. The experimental results reveal that LLM-based prediction methods achieve exceptional compression rates, exceeding 20×, which significantly outperforms the 3× achieved by the state-of-the-art lossless compressor Gzip. Moreover, we demonstrate that the superior performance of LLM-based compressors generalizes across different LLM scales and diverse dataset categories, making them a useful tool in lossless data compression.

1 Introduction

The proliferation of large language models (LLMs) has led to a significant surge in AI-generated text across diverse domains [6, 13, 42, 44]. Given proper instructions from humans, LLMs can generate desirable, high-quality text for multiple purposes. The great capability and versatility of LLMs have made the use and deployment of LLMs continue to expand, which contributes an exponentially growing proportion of the text data in the world. OpenAI’s ChatGPT reportedly produces approximately 100 billion words daily [43]. In academia, an estimated 60,000 scholarly articles in 2023—over 1% of all publications—were likely written with LLM assistance [20], with 17.5% of newly published computer science papers and 16.9% of peer review texts incorporating LLM-generated content [28]. An analysis of over 6 billion sentences from Common Crawl further revealed that more than 57% were machine-translated, highlighting the extent of automated content generation [47]. In fact, LLMs have also influenced various fields, including creative writing, healthcare,

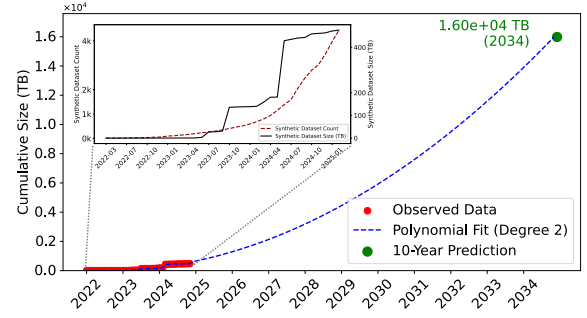


Figure 1: Trend of increasing LLM-generated data.

law, and software development. Authors use LLMs to co-author novels and generate narratives, as seen in Ayad Akhtar’s play “McNeel” [1]. In healthcare, they aid in diagnostics, automate documentation, and enhance medical education [50]. In the legal domain, LLMs assist in drafting legal documents and summarizing cases [7]. Software development has also benefited from LLM-powered code generation, streamlining programming workflows [8]. In one of the largest dataset host platforms – Huggingface,¹ we find that the LLM-generated data grows at an exponential speed, as shown in Fig. 1. Furthermore, we fit a quadratic polynomial regression model, which predicts that the LLM-generated data will continue to expand up to 16,000 TB within the next decade. The exponential growth of LLM-generated text poses significant demands and challenges for efficiently storing LLM-generated data.

The rise of LLMs has introduced a new category of machine-generated data: *LLM-generated text*. Prior to the era of LLMs, machine-generated data usually referred to structured texts such as logs [5, 26]. For example, an IoT device can generate up to 1 TB per day in industrial applications, which are typically structured, containing numerical values and categorical information [53]. As machine-generated text grows at a much higher speed than that from humans, a number of traditional compressors are designed to reduce the data storage volume of machine-generated text through the exploitation of the intrinsic structures [52, 56, 57]. In contrast, LLM-generated text shares the trait of even more significantly rapid growth but exhibits distinct characteristics.

Therefore, in this work, we conduct the first investigation of effective lossless compression approaches for LLM-generated data. We begin with an in-depth analysis of the characteristics and compressibility of LLM-generated synthetic datasets. Specifically, we

¹<https://huggingface.co/>

consider and curate 3 LLM-generated synthetic datasets from different domains. Based on those datasets, we measure the redundancy levels in LLM-generated text through n-gram statistics (Fig. 2), and find it is challenging to simply deduplicate the LLM-generated text. Furthermore, we assess the compression potential by examining entropy per byte and mutual information (Table 2), which shows that LLM-generated data share similar levels of redundancy and information density to human-generated data. Consequently, we find that traditional compression approaches, such as non-neural and neural compressors, can not achieve satisfactory compression ratios (Table 3). Most methods achieve only a compression ratio of 3–6×, including entropy compression techniques based on neural networks. Our findings provide an empirical foundation for designing efficient compression techniques *tailored to the unique characteristics* of LLM-generated text.

To this end, we propose to incorporate LLMs to compress LLM-generated text. As LLMs are trained through next-token prediction, LLMs demonstrate high predictability on the LLM-generated data. Hence, we can leverage the inherent predictability of LLMs to build powerful lossless compressors for LLM-generated data with remarkable compression capabilities. The need for *compression rather than re-generation* arises from the randomness in LLM generation— even with the same prompt and model, exact re-generation remains infeasible. Therefore, an LLM can serve as a strong predictor for other LLM-generated data, but not with absolute accuracy. We conduct extensive experiments using 14 representative LLMs and 8 LLM-generated datasets spanning diverse categories. The results demonstrate that LLM-based prediction methods achieve exceptional compression rates, exceeding 20×, which significantly surpasses the 3× compression ratio achieved by Gzip, the state-of-the-art lossless compression algorithm. Furthermore, we show that the superior performance of LLM-based compressors generalizes across various LLM scales and dataset categories, highlighting their robustness and potential as a valuable tool for lossless data compression. Finally, we also conduct an in-depth analysis of factors influencing compression efficiency, such as chunk size and data scale.

To summarize, this paper makes the following contributions:

- To the best of our knowledge, we present the *first systematic analysis* of the characteristics of LLM-generated data in terms of compressibility. As an emerging new category of machine-generated data, LLM-generated data exhibits significantly different characteristics from traditional machine-generated data while being similar to human-generated data. Consequently, traditional compressors can not effectively compress LLM-generated data.
- We are also the first to propose leveraging LLMs to compress LLM-generated synthetic data. We show that LLMs have high inherent predictability for the LLM-generated data, inherited from the training with next-token prediction, despite differences in the training sources. It allows LLMs to compress the LLM-generated data effectively.
- We adopt 8 datasets from different domains, and 14 state-of-the-art LLMs of different scales, trained on diverse data

sources, and demonstrate that LLM-based compression consistently achieves remarkable compression rates. *While conventional methods achieve only 3–5× compression, LLMs can reach nearly 23×.* We also conduct an in-depth analysis of the influence of various factors, such as different models, datasets, data sizes, and chunk sizes. Across various model sizes, datasets, data sizes, and chunk sizes, LLM-based compressors maintain significantly higher compression ratios than previous methods.

We hope our findings provide new insights and foundations for the management of text data in large-scale database.

2 Background

In this section, we begin by defining LLM- LLM-generated data and comparing it to conventional machine-generated data, emphasizing their core distinctions. We then examine the key characteristics of LLM-generated text in contrast to other datasets. We summarize the characteristics of different types of data in Table 1. Finally, we delve into the role of randomness in LLM text generation, illustrating how stochastic elements within the model contribute to both diversity and uncertainty in the resulting outputs.

Table 1: Comparison of Machine-Generated Data, LLM-Generated Data, and Human-Generated Data.

Characteristic	Machine-Gen	LLM-Gen	Human-Gen
Automatically Generated	✓	✓	–
Mimics Human Writing	–	✓	✓
High Volume	✓	✓	–
Highly Structured	✓	–	–
Time-Series Nature	✓	–	–
Contains Semantic Context	–	✓	✓
Factual & Deterministic	✓	–	–

2.1 Conventional Machine-Generated Data

Machine-generated data refers to data that is entirely produced by machines or software without human intervention. The generation process is usually automated, occurring at fixed intervals or when triggered by specific events. The core characteristic of machine-generated data is that it originates from computational processes or hardware systems rather than being manually recorded. Examples include system logs, sensor readings, network traffic, and financial records. These data types are automatically generated by computational systems, IoT devices, and network infrastructure, often capturing real-time events and transactions without direct human input. They serve critical roles in monitoring, diagnostics, security analysis, and operational decision-making across industries [53].

2.2 Large Language Model-Generated Data

Synthetic data refers to data that is not derived directly from real-world events but is instead artificially generated by algorithms. LLM-generated synthetic data specifically refers to the text data created by Large Language Models (LLMs). Unlike traditional machine-generated data, which follows predefined rules, LLM-generated data mimics human language patterns, maintains contextual coherence,

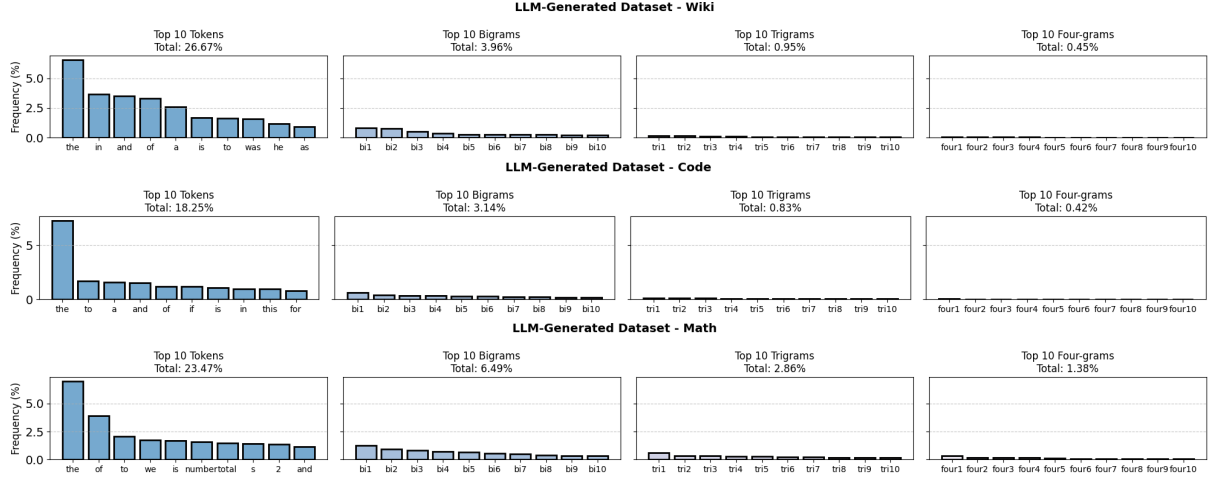


Figure 2: N-gram frequency count for LLM-generated synthetic data.

and exhibits linguistic diversity. Trained on massive corpora, LLMs generate text that closely resembles human writing, making them valuable for applications in content creation, data augmentation, and knowledge synthesis [1, 7, 8, 20, 50].

Code-related datasets dominate with 571 datasets, aligning with the growing importance of code generation and LLMs in software development. Linguistics follows with 471 datasets, reflecting the widespread use of synthetic data in NLP tasks like text generation and translation. Other major categories include art (432 datasets), literature (151 datasets), and music (151 datasets), highlighting AI’s role in creative content generation. Science (246 datasets), math (165 datasets), and education (226 datasets) emphasize knowledge-based applications, while medical (131 datasets), business (38 datasets), finance (49 datasets), and law (77 datasets) demonstrate the increasing use of synthetic data in professional and regulatory fields.

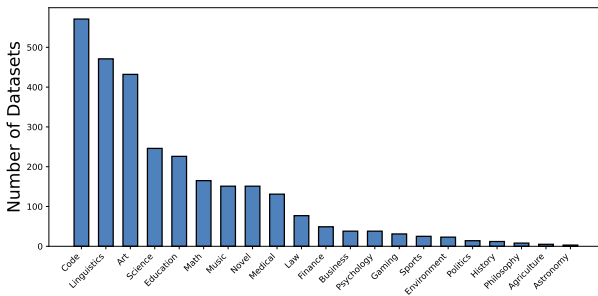


Figure 3: LLM-generated data distribution.

2.3 Characteristics of LLM-Generated Text vs. Other Datasets

We begin by analyzing the characteristics of different types of data, aiming to gain insights that can guide more effective compression strategies. The characteristics of LLM-generated text differ significantly from traditional structured datasets used in database

benchmarks. While machine-generated datasets such as TPC-H [11] are designed with well-defined schemas and structured query workloads, LLM-generated text exhibits high variability and redundancy and lacks explicit tabular organization. Similarly, human-generated text data—such as web articles, social media content, and historical corpora—shares some properties with LLM-generated text.

Table 1 compares the key characteristics of machine-generated data, LLM-generated data, and human-generated data. Machine-generated data is highly structured, deterministic, and often time-series-based, whereas LLM-generated data mimics human writing and lacks the strict structure of traditional machine-generated formats. Unlike human-generated data, LLM-generated data is automatically produced and exhibits high volume but may not always be factually accurate.

3 Compressibility Analysis of LLM-Generated Data

In this subsection, we begin by exploring the potential for deduplication in LLM-generated data. We then evaluate the information content of machine-generated, human-generated, and LLM-generated data using entropy on different levels of tokenization and mutual information. Finally, we discuss the limitations of existing compression methods, setting the stage for new strategies that leverage the predictability of LLM-generated data.

3.1 Deduplication Potential of LLM-Generated Data

Deduplication is a common technique for machine-generated data, such as tabular or time-series data [52, 56]. To assess its applicability to LLM-generated data, we performed an n-gram analysis on LLM-generated synthetic datasets to evaluate their redundancy and deduplication potential. By examining the frequency distribution of tokens, bigrams, trigrams, and four-grams across different domains, we aim to obtain a holistic view of the redundancy of these datasets, i.e., how much repeated content exists in these datasets

and whether deduplication could effectively reduce storage or improve compression efficiency.

As illustrated in Fig. 2, the results reveal a low level of redundancy across all datasets. In the clinical dataset, for example, the top 10 tokens account for 27.53% of the text, but bigrams, trigrams, and four-grams contribute significantly less, with bigrams at 5.03%, trigrams at 1.86%, and four-grams at just 0.93%. A similar pattern is observed in the code and math datasets, where repeated n-grams take up only a small portion of the text.

These findings suggest that deduplication techniques, which have good performance on machine-generated data, have limited potential for LLM-generated synthetic datasets. Unlike conventional machine-generated data, which often contains repetitive phrases and structural redundancies, LLM-generated text is more diverse, reducing the effectiveness of deduplication-based compression methods. One possible scenario in the future is that when the same LLM generates text from different sources, there may be opportunities for file-level deduplication. However, this falls beyond the scope of the research topic discussed in this paper.

3.2 Information Evaluation of the machine-generated data, human-generated data, and LLM-generated data.

Lossless compression aims to encode a sequence of symbols $x_{1:n}$ into a bitstream with minimal expected length while ensuring perfect reconstruction of the original data. According to Shannon's source coding theorem [45], the optimal compression efficiency is bounded by the entropy of the source distribution. Given a source probability distribution $\rho(x)$, the Shannon entropy is defined as:

$$H(\rho) := \mathbb{E}_{x \sim \rho} [-\log_2 \rho(x)] \quad (1)$$

The Shannon entropy represents the theoretical lower bound on the average number of bits needed to encode a symbol drawn from ρ . A well-designed compression scheme assigns shorter codes to more frequent symbols and longer codes to rare symbols, achieving an average code length L satisfying:

$$L \geq H(\rho) \quad (2)$$

where equality holds under an optimal entropy coder such as Huffman coding or Arithmetic coding in the limit of large sequences. This entropy bound establishes the fundamental limits of lossless data compression, beyond which further compression requires exploiting higher-order dependencies or introducing loss.

To evaluate the impact of tokenization granularity on information representation, we compute the entropy per byte for character-level, subword (BPE), and word-level tokenization. The entropy per byte is computed as follows:

$$H_{\text{byte}} = \frac{H_{\text{token}}}{L_{\text{avg}}}$$

where H_{token} is the Shannon entropy per token, calculated as:

$$H_{\text{token}} = - \sum_{t \in T} P(t) \log_2 P(t)$$

where T is the set of tokens, and $P(t)$ is the probability of token t . L_{avg} is the average token length in bytes, defined as:

$$L_{\text{avg}} = \frac{\sum_{t \in T} P(t) \cdot L(t)}{\sum_{t \in T} P(t)}$$

where $L(t)$ is the byte length of token t . This formula applies to character-level, subword (BPE), and word-level tokenization by adjusting the token set T accordingly.

We also use *Mutual Information Between Consecutive Words*, which measures how much knowing one word helps predict the next word. It is computed as:

$$MI(W_i, W_{i+1}) = \sum_{w_i \in W} \sum_{w_{i+1} \in W} P(w_i, w_{i+1}) \log_2 \frac{P(w_i, w_{i+1})}{P(w_i)P(w_{i+1})}$$

where W_i and W_{i+1} are consecutive words in the dataset, $P(w_i, w_{i+1})$ is the joint probability of two consecutive words w_i and w_{i+1} , and $P(w_i)$ and $P(w_{i+1})$ are the marginal probabilities of words w_i and w_{i+1} , respectively. This metric provides insights into the structural dependencies within different tokenization schemes, helping assess how well each method retains contextual information.

Table 2 presents a comparative analysis of these characteristics, where Char-E, BP-E, and W-E indicate Char-Entropy, BPE-Entropy and Word-Entropy. To ensure a fair comparison, the human-generated dataset and the LLM-generated dataset both originate from the same source (Wikipedia), with each sentence conveying the same meaning. The only difference is that the LLM-generated data has been rewritten by an LLM. For the machine-generated data, i.e., TPC-H, we collected the comments field from its tables to ensure a fair comparison with the other two datasets.

Table 2: Redundancy and token frequency characteristics of LLM-generated text vs. structured datasets.

Dataset	Char-E	BP-E	W-E	Mutual Info
LLM-Generated	4.67	2.51	2.57	2.95
Human-Generated	4.63	2.53	2.59	2.73
TPC-H	4.34	1.72	0.94	1.23

We observe that the entropy measures of both LLM-generated data and human-generated data are similar across character, BPE, and word levels. Moreover, these entropy values are relatively low, indicating that traditional compression methods, including state-of-the-art approaches like Zstd, are likely to achieve only relatively limited compression ratios.

Interestingly, when examining mutual information (Mutual Info), LLM-generated data exhibits relatively higher token predictability, despite each sentence conveying the same meaning as its human-generated counterpart. As expected, TPC-H shows very low mutual information since it is randomly generated and lacks a continuous semantic structure.

3.3 Limitations of Existing Compression Methods

Built upon the aforementioned analysis, we selected three types of LLM-generated synthetic text for a preliminary examination of traditional compression methods, including a mixed-domain

Wikipedia dataset and two specific-domain datasets: math and code. For compression baselines, we used three state-of-the-art compressors—LZMA [58], Gzip [15], and Zstd [10], along with three Neural Network-based (NN-based) compressors—NNCP [3], TRACE [34], and PAC [35] to conduct a comprehensive study on the compressibility of LLM-generated text.

Table 3: Compression ratios of different datasets using Gzip, LZMA, Zstd-22, NNCP, TRACE and PAC.

Dataset	Gzip	LZMA	Zstd	NNCP	TRACE	PAC
Wiki	2.73	4.24	4.17	5.04	4.11	4.17
Code	3.47	4.90	4.85	6.30	4.53	5.08
Math	4.30	5.99	5.90	7.76	5.34	6.39

The compression results are given in Table 3. We can find that, while modern compressors such as LZMA and neural-based methods like NNCP and PAC outperform traditional dictionary-based approaches such as Gzip and Zstd-22, the overall compression ratios remain relatively limited. Among the traditional methods, LZMA achieves the highest compression efficiency, with ratios ranging from 4.24 (Wiki) to 5.99 (Math). Zstd-22 performs similarly to LZMA but offers slightly lower compression ratios across all datasets. Neural-based methods demonstrate further improvements, with NNCP consistently achieving the best performance among the listed approaches, obtaining 6.30 (Code), 7.76 (Math), and 5.04 (Wiki). TRACE and PAC provide competitive results but still fall within a similar compression range.

Despite these incremental improvements, all existing methods struggle to achieve compression ratios beyond a single-digit range. The results highlight the inherent limitations of conventional entropy and dictionary-based compressors, as well as the current generation of neural compressors, when applied to unstructured and irregular text datasets. It calls for developing efficient compression techniques tailored to the unique characteristics of the LLM-generated text.

4 LLM-based Compression Framework

To resolve the irregularity of the LLM-generated text, we propose to leverage LLMs to compress the LLM-generated text. In this section, we present a detailed LLM-based framework that achieves remarkable lossless compression performance compared to traditional compression approaches.

4.1 Conceptual Framework

In this section, we provide a conceptual description of our LLM-based compression framework. It consists of two steps: a) context-based prediction and b) encoding coding.

4.1.1 Context-based prediction. Context-based prediction is a foundational technique in neural lossless compression[3, 14, 34], aiming to estimate the probability distribution of a symbol x_t conditioned on its preceding context $x_{<t}$. Accurate probability estimation affords efficient entropy coding and reduces the redundancy in data representation.

Given a sequence of symbols $X = (x_1, x_2, \dots, x_T)$ in length T , the probability distribution of the entire sequence can be factorized

using the chain rule of probability:

$$P(X) = \prod_{t=1}^T P(x_t | x_{<t}), \quad (3)$$

where $x_{<t} = (x_1, x_2, \dots, x_{t-1})$ represents the preceding symbols. The objective of context-based prediction is to accurately model the conditional probability $P(x_t | x_{<t})$ to minimize the entropy of the encoded sequence.

4.1.2 Entropy coding. Entropy coding relies on the fundamental principle that the optimal encoding length of a symbol x_t is given by its self-information:

$$L(x_t) = -\log_2 P(x_t | x_{<t}). \quad (4)$$

Thus, the expected length per symbol, also known as the entropy, is given by:

$$\begin{aligned} H(X) &= \mathbb{E}[-\log_2 P(x_t | x_{<t})] \\ &= -\sum_{x_t} P(x_t | x_{<t}) \log_2 P(x_t | x_{<t}). \end{aligned} \quad (5)$$

Efficient entropy coding is achieved when the estimated probability distribution $\hat{P}(x_t | x_{<t})$ closely matches the true probability distribution $P(x_t | x_{<t})$, leading to minimal redundancy in encoding. On the other hand, the coding inefficiency R can be quantified as:

$$R = H(\hat{P}) - H(P), \quad (6)$$

where $H(\hat{P})$ is the entropy computed using the estimated probability distribution. The Kullback-Leibler (KL) divergence provides a measure of the discrepancy between these distributions:

$$D_{KL}(P \parallel \hat{P}) = \sum_{x_t} P(x_t | x_{<t}) \log_2 \frac{P(x_t | x_{<t})}{\hat{P}(x_t | x_{<t})}. \quad (7)$$

From information theory, it follows that:

$$H(P) \leq H(\hat{P}) = H(P) + D_{KL}(P \parallel \hat{P}). \quad (8)$$

This inequality implies that a more accurate probability model reduces D_{KL} , leading to a smaller gap between the estimated entropy $H(\hat{P})$ and the true entropy $H(P)$, thereby improving compression efficiency. In the ideal case where $\hat{P} = P$, the KL divergence becomes zero, achieving optimal entropy coding.

4.2 The Next-Token Prediction Framework

LLMs generate text in an autoregressive manner. In other words, LLMs predict the next token in a sequence based on previously generated tokens. The training of LLMs leverages the next-token prediction objective, which is to model the probability distributions over sequences of tokens.

The training process of LLMs essentially resembles the context-based prediction. To capture relevant contextual information, LLMs employ an embedding function f_θ , parameterized by θ , to transform the context $x_{<t}$ into a hidden state representation h_t :

$$h_t = f_\theta(x_{<t}). \quad (9)$$

Transformer-based Models (e.g., GPT-like Architectures) utilize self-attention mechanisms to capture long-range dependencies [49]:

$$h_t = \sum_{j<t} \alpha_{t,j} (W_V x_j), \quad (10)$$

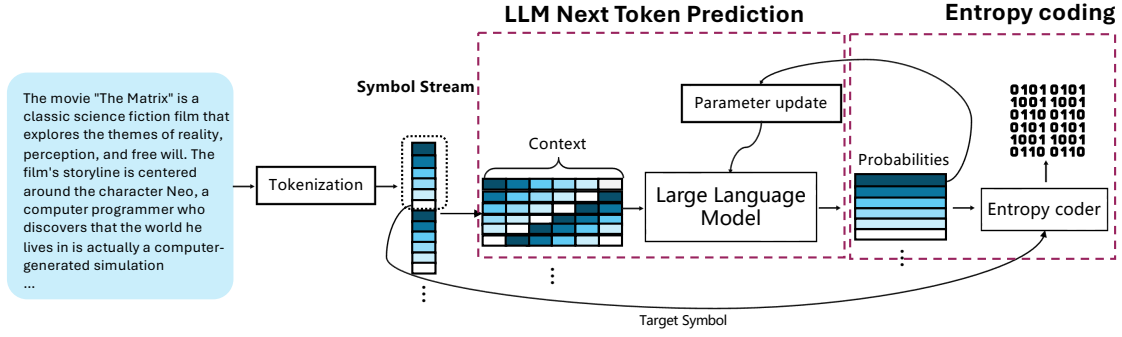


Figure 4: General process of Neural-based Compression.

where attention weights $\alpha_{t,j}$ are computed as follows:

$$\alpha_{t,j} = \frac{\exp((W_Q x_t)^\top (W_K x_j))}{\sum_{j < t} \exp((W_Q x_t)^\top (W_K x_j))}. \quad (11)$$

Once the contextual representation h_t is obtained, the probability distribution of the next token x_t is modeled using a softmax function:

$$P(x_t | x_{<t}) = \text{softmax}(W_o h_t), \quad (12)$$

where W_o is a learnable weight matrix that maps the hidden state to output logits. The predicted probability distribution is then utilized in entropy coding algorithms to achieve optimal lossless compression. In this work, we implement the entropy coding as the arithmetic coding [51].

Tokenization and Embedding. Prior to processing, text is tokenized into discrete units using methods such as Byte Pair Encoding (BPE) or WordPiece. Each token x_t is then mapped to a high-dimensional embedding vector e_t via a pre-trained embedding matrix:

$$e_t = \text{Embedding}(x_t). \quad (13)$$

Transformer-Based Processing. The token embeddings are processed through multiple layers of a Transformer model, which consists of self-attention and feedforward networks. The self-attention mechanism computes token dependencies using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}} + M\right)V, \quad (14)$$

where Q , K , and V are query, key, and value matrices, respectively, and M is a causal mask preventing future tokens from influencing current token computations.

Probability Estimation and Decoding. The final hidden representation h_t of the last Transformer layer is projected onto the vocabulary space and converted into a probability distribution using a softmax function:

$$P(x_{t+1}|x_1, \dots, x_t) = \text{softmax}(Wh_t + b), \quad (15)$$

where W and b are trainable parameters.

Training Objective. The model is trained using a causal language modeling objective, or next-token prediction, minimizing the cross-entropy loss:

$$L = - \sum_{t=1}^T \log P(x_t | x_1, \dots, x_{t-1}). \quad (16)$$

Gradients are computed via backpropagation, and model parameters are updated using optimization algorithms such as Adam [29].

4.3 Entropy Coding

We use arithmetic coding as the entropy coder [51]. Arithmetic coding is an advanced entropy encoding technique that efficiently represents a sequence of symbols as a single floating-point number within the unit interval $[0, 1)$. Unlike Huffman coding, which assigns fixed-length binary codes to individual symbols [24], arithmetic coding progressively refines an interval based on symbol probabilities, achieving near-optimal compression close to the Shannon entropy limit.

4.3.1 Encoding Process. Given an input sequence $X = (x_1, x_2, \dots, x_T)$ in length T with respective probabilities $P(x_t)$, arithmetic coding proceeds as follows. Initially, the encoding interval is set to $[L, H) = [0, 1)$.

For each symbol x_t in the sequence, the current interval width is computed as:

$$R = H - L.$$

The interval is then updated based on the cumulative probability of preceding symbols:

$$L' = L + R \cdot F(x_t), \quad H' = L + R \cdot (F(x_t) + P(x_t)),$$

where $F(x_t)$ represents the cumulative probability of all symbols preceding x_t .

This process continues iteratively until all symbols have been encoded. The final representation is chosen as a value within the interval $[L_T, H_T)$, ensuring a minimal binary representation.

4.3.2 Decoding Process. Decoding follows the inverse procedure. The interval is initialized as $[L, H) = [0, 1)$, and the encoded value X is read.

At each step, the symbol x_t is identified as the one whose assigned sub-interval $[L', H')$ contains X . The decoded symbol is then

output, and the interval is updated using the same transformation as in the encoding process.

To ensure correct symbol retrieval in subsequent steps, X is normalized within the updated interval, and the process continues until the entire sequence is reconstructed.

4.4 Compression or Generation?

One might be curious about the necessity of using LLMs to compress LLM-generated instead of directly regenerating the text. In most cases, LLMs do not generate identical outputs for the same input due to multiple interacting factors that introduce variability. While LLMs are fundamentally deterministic in that they map a given input sequence to a set of probability distributions over tokens, the presence of stochastic sampling, architectural routing mechanisms, and computational nuances can lead to nondeterministic behavior. Even when all explicit sources of randomness, such as temperature and sampling, are eliminated, underlying numerical precision issues and architectural decisions may still cause slight variations in the generated text.

Temperature. One of the primary factors contributing to output variability is the temperature (τ), a parameter that modulates the probability distribution over possible next tokens. Mathematically, temperature scales the logits (z_i) corresponded to word (w_i), before applying the softmax function:

$$P(w_i) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}. \quad (17)$$

When $\tau > 1$, the probability distribution becomes flatter, increasing the likelihood of selecting lower-probability tokens, which enhances diversity in the generated output. Conversely, as $\tau \rightarrow 0$, the softmax function increasingly favors the token with the highest probability, making the output more deterministic [22]. However, due to floating-point precision limitations, even a near-zero temperature may not always yield perfectly consistent results.

Sampling. In addition to temperature, sampling strategies also significantly influence output determinism. Techniques such as top- k sampling restrict token selection to the k most probable candidates [17], while top- p (nucleus) sampling dynamically adjusts the candidate set to include the smallest subset of tokens whose cumulative probability exceeds a threshold p [22]. Additionally, even deterministic decoding methods such as greedy decoding (where the model selects the most probable token at each step) or beam search [18] (which maintains multiple hypotheses before selecting the highest-probability sequence) can exhibit variations due to numerical rounding errors, computational parallelism, and subtle differences in implementation.

Architecture. The model’s architecture can also cause the issue of non-determinism. In traditional dense Transformer models, such as GPT-3 [4] and LLaMA [48], every token undergoes a fixed computational pathway, ensuring a higher degree of determinism when sampling is disabled [48]. However, more recent models, such as GPT-4 [44], are believed to employ Mixture-of-Experts (MoE) architectures, where only a subset of specialized neural network components, or “experts” are activated for each token [27]. The routing mechanism that determines which experts are used can introduce

variability for its probabilistic nature, which can be sensitive to minute numerical fluctuations [16].

5 Evaluation

In this section, we systematically evaluate the compression performance of various methods across multiple LLM-generated datasets. We begin by outlining the experimental setup. Next, we introduce the baseline methods, including traditional entropy-based, dictionary-based, and NN-based compression techniques, providing a comprehensive comparison against our proposed approach.

To better understand key factors influencing compression efficiency, we conduct in-depth analyses on chunk size, model size, and dataset scale, investigating their impact on compression ratios. Additionally, we compare LLMs with and without instruction tuning, exploring how post-training affects LLM-based compression. Finally, we categorize the data based on structural properties and statistical characteristics, offering insights into how different types of text influence compression efficiency.

5.1 Experimental Setups

In this section, we detail the datasets, preprocessing steps, and evaluation metrics used in our study.

5.1.1 Evaluation Datasets.

Wiki [41]. This dataset consists of human-written and LLM-generated Wikipedia introductions. The human-written samples are sourced from Wikipedia, while the machine-generated samples are produced using the GPT-3 model.

Article [41]. The Article dataset includes scientific research abstracts in both human-written and machine-generated formats. Human-written abstracts are sourced from existing scientific publications, while LLM-generated abstracts are produced using GPT-3.5.

Code [54]. This dataset was generated using Mixtral-8x7B. It contains programming problems and solutions across multiple languages, including Python, JavaScript, TypeScript, C++, and C.

Math [39]. The Math dataset contains approximately 200K grade school math word problems, designed to improve the mathematical reasoning capabilities of language models. The answers were generated using Azure GPT-4-Turbo, ensuring consistency across different types of problems.

Science [31]. The Science dataset consists of 20K problem-solution pairs across 25 physics topics, with 25 subtopics per topic and 32 problems per topic-subtopic pair. All problem-solution pairs were generated using GPT-4, making this dataset well-suited for evaluating the compression of structured scientific content.

Clinical [30]. This dataset is the official dataset for Asclepius (arXiv), formatted in a Clinical Note - Question - Answer structure. Synthetic clinical notes were generated from PMC-Patient case reports using GPT-3.5, followed by instruction-answer pairs for 157K synthetic discharge summaries.

Web. The Web dataset consists of movie critic reviews generated using ChatGPT [43], structured to mimic human-written reviews.

Table 4: Detailed Characteristics of Selected Large Language Models

Model	Size (B)	Type	Year	Organization	Context Length	Data Source	Use Case
OpenELM-1.1B [36]	1.1B	Decoder	2024	Apple	4096	Web, Books	Efficient NLP tasks
AMD-OLMo-1B [33]	1B	Decoder	2024	AMD	4096	Code, Web Data	Optimized AI applications
Llama-3.2-1B [38]	1B	Decoder	2024	Meta AI	8192	Web, Academic Papers	General NLP
Llama-3.2-1B-Instruct [38]	1B	Decoder	2024	Meta AI	8192	Web, Academic Papers	Instruction-tuned NLP
Llama-3.2-3B [38]	3B	Decoder	2024	Meta AI	8192	Web, Books, Papers	Open-source NLP
Llama-3.2-3B-Instruct [38]	3B	Decoder	2024	Meta AI	8192	Web, Books, Papers	Instruction-tuned NLP
Llama-3.1-8B [37]	8B	Decoder	2024	Meta AI	16384	Large-scale Corpora	Advanced reasoning
Llama-3.1-8B-Instruct [37]	8B	Decoder	2024	Meta AI	16384	Large-scale Corpora	Instruction-following NLP
Qwen2.5-14B [46]	14B	Decoder	2024	Alibaba	32768	Multilingual Corpora	General NLP
Qwen2.5-14B-Instruct-1M [46]	14B	Decoder	2024	Alibaba	32768	Multilingual Corpora	Instruction-tuned NLP
Qwen2.5-Math-1.5B [55]	1.5B	Decoder	2024	Alibaba	16384	Mathematical Data	Mathematical reasoning
Rho-Math-1B [32]	1B	Decoder	2024	Microsoft	2048	Mathematical Data	Mathematical problem
Qwen2.5-Coder-1.5B [25]	1.5B	Decoder	2024	Alibaba	16384	Code Corpora	Code generation
DeepSeek-Coder-1.3b-Instruct [21]	1.3B	Decoder	2024	DeepSeek AI	16000	Code, Natural Language	Code completion

Novel. The Novel dataset consists of a travel book generated using LongWriter [2], simulating the style and structure of human-authored long-form narratives.

5.1.2 Evaluation Metrics. For lossless compression, the compression ratio is the primary performance metric. It quantifies the reduction in data size achieved by compression and is defined as the ratio between the original data size and the compressed data size:

$$R = \frac{S_{\text{original}}}{S_{\text{compressed}}} \quad (18)$$

where S_{original} is the size of the uncompressed data, and $S_{\text{compressed}}$ is the size of the data after applying a compression algorithm. A higher compression ratio indicates greater space savings, making storage and transmission more efficient.

5.1.3 Hardware Platform. Our experiment is conducted on a cluster powered by an AMD EPYC 7742 64-Core Processor equipped with a NVIDIA A100 Tensor Core GPU.

5.2 Baselines

To thoroughly investigate the compression characteristics of LLM-generated data, we introduce nine baselines spanning three categories: entropy-based, dictionary-based, and neural-based methods. Additionally, to examine the impact of different LLMs on the proposed framework, we include 14 LLMs in this comprehensive study.

5.2.1 Entropy-based Compressor. Huffman coding, Arithmetic coding and Finite State Entropy (FSE) is selected as entropy-based baselines. Huffman coding [24] is a foundational entropy coding technique, assigns shorter codes to more frequent symbols and serves as the basis for many compression algorithms. Finite State Entropy (FSE) [9] is an entropy coding method that dynamically models symbol probabilities using a finite-state machine, improving efficiency over fixed-length coding schemes like Huffman coding. Arithmetic coding [51] represents an entire message as a fractional number in the interval $[0, 1)$, refining the interval based on symbol probabilities to achieve near-optimal compression.

5.2.2 Dictionary-based Compressor. LZMA (Lempel-Ziv-Markov chain algorithm) [58] extends LZ77 with probability modeling and range coding, achieving high compression ratios at the cost of

increased computational complexity. Gzip [15], based on the DEFLATE algorithm, integrates LZ77 for dictionary compression with Huffman coding for entropy encoding, offering a well-balanced tradeoff between compression ratio and speed, widely used for file compression and web data transfer. Zstandard (Zstd) [10], developed by Facebook, is a high-performance compression algorithm that combines LZ77-style compression with Finite State Entropy (FSE) coding, balancing speed and compression ratio while supporting dictionary-based compression.

5.2.3 NN-based Compressor. We adopt NNCP [3], TRACE [34] and PAC [35] as our NN baselines. Neural Network-based Compression (NNCP) utilizes deep learning models, such as autoencoders and transformers, to learn compact representations of data. TRACE designed a slim transformer to accelerate the compression procedure. PAC further builds an order model to replace attention to achieve faster compression speed.

5.2.4 LLM-based Compressor. In this paper, we incorporate LLMs of different sizes and from various companies, covering a range from 1B to 14B parameters. In deep learning, 250M, 1B, and 11B refer to models with 250 million, 1 billion, and 11 billion parameters, respectively. Each parameter is typically stored as a 16-bit (FP16) or 32-bit (FP32) floating-point number.

The selected models provide a diverse comparison across different model sizes and organizations. Specifically, we included multiple 1B-sized models, OpenELM-1.1B, AMD-OLMo-1B, and Llama-3.2-1B, to evaluate performance under similar computational constraints. The Llama series was chosen to investigate scaling laws in compression tasks, given its variations in size and training methodology. Furthermore, models from various companies, including Apple, AMD, Meta AI, and Alibaba, were selected to assess the generalization of our proposed method across different architectures and datasets.

Additionally, we include instruction-tuned versions of several models, which are aligned to human preferences, including Llama-3.2-1B-Instruct, Llama-3.2-3B-Instruct, Llama-3.1-8B-Instruct, and Qwen2.5-14B-Instruct-1M. Compared to their base counterparts, these models demonstrate enhanced usability in real-world interactive AI applications. In the following experiments, unless otherwise specified, we use Llama-3.1-8B as the base LLM.

Table 5: Compression results for various LLM-generated synthetic datasets.

Method	Entropy-based Compressor							
	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
Huffman	1.69	1.62	1.65	1.82	1.77	1.64	1.76	1.81
Arithmetic	1.70	1.63	1.66	1.61	1.77	1.64	1.77	1.81
FSE	1.68	1.62	1.65	1.83	1.77	1.64	1.75	1.80
Method	Dictionary-based Compressor							
	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
Gzip	2.73	3.47	4.30	5.70	4.26	5.20	3.40	3.48
Lzma	4.24	4.90	5.99	9.76	6.49	7.13	5.74	5.34
Zstd-22	4.17	4.85	5.90	9.95	6.46	7.05	5.68	5.31
Method	Neural-based Compressor							
	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
NNCP	5.04	6.30	7.76	12.56	2.95	7.38	5.39	6.30
TRACE	4.11	4.53	5.34	9.55	4.75	5.09	5.09	5.53
PAC	4.17	5.08	6.39	9.94	3.48	6.66	5.02	5.63
Ours	14.62	20.44	15.22	17.84	22.26	23.80	23.13	16.97

5.3 Overview of Compression Performance Across LLM-Generated Datasets

In this subsection, we provide a comprehensive analysis of compression efficiency across various LLM-generated synthetic datasets. Table 5 presents the results obtained using different categories of compression methods, including entropy-based, dictionary-based, and neural-based approaches. The evaluation covers a diverse range of datasets, including Wiki, Code, Math, Clinical, Web, Science, Novel, and Article, allowing for a thorough comparison of how different methods handle distinct types of LLM-generated text.

The entropy-based compressors, including Huffman coding, Arithmetic coding, and FSE coding, exhibit the lowest compression performance across all datasets. Their limited ability to exploit long-range dependencies in text restricts their effectiveness, resulting in compression ratios consistently below 2.0. This highlights the fundamental limitation of statistical coding techniques when applied to highly structured, synthetic text.

Dictionary-based methods, such as Gzip, LZMA, and Zstd-22, show a significant improvement over entropy-based approaches, with compression ratios ranging from approximately 2.7 to 9.9 across different datasets. Among them, LZMA consistently achieves the highest compression rates, particularly on structured datasets such as Clinical and Science, where it benefits from repetitive patterns. However, despite their improved efficiency, these methods still fall short of neural compression approaches in leveraging deeper contextual information.

Neural-based compressors, including NNCP, TRACE, and PAC, further enhance compression performance, demonstrating that learned models can better capture the statistical structure of LLM-generated text. NNCP achieves particularly high compression rates on complex datasets such as Math and Science, while PAC and TRACE show competitive results across most categories. Notably, neural approaches tend to perform better on more structured datasets, suggesting that their learned representations effectively exploit latent regularities in LLM-generated content.

The most striking result comes from the proposed method, which significantly outperforms all baseline compressors across every

dataset. The compression rates achieved by our approach are consistently higher than those of both traditional and neural-based compressors, demonstrating its superior ability to model LLM-generated data. Notably, the largest improvements are observed on Science, Clinical, and Novel datasets, where compression rates surpass 20. This indicates that our method effectively captures and exploits the structured patterns inherent in LLM-generated text, leading to unprecedented gains in compression performance.

5.4 Impact of Chunk Size on LLM Compression Performance

Chunk size significantly influences LLM compression performance by shaping how contextual information is utilized for token prediction. This subsection examines the effect of varying chunk sizes across different model configurations, including both base and instruction-tuned variants. Specifically, we test Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B, along with their instruction-tuned counterparts, across chunk sizes ranging from 16 to 256.

Experimental results for Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B, along with their instruction-tuned counterparts, consistently show that larger chunk sizes yield higher compression rates. This aligns with the theoretical premise that a broader context enables more accurate token predictions, reducing redundancy in the encoded representation. Compression efficiency improves progressively from chunk size 16 to 256, with larger models such as Llama-3.1-8B exhibiting steeper gains. Instruction-tuned models systematically outperform their base counterparts, particularly at larger chunk sizes, indicating that instruction tuning enhances token predictability. However, compression gains diminish beyond a certain threshold. The most notable improvements occur between chunk sizes 16 and 64, while gains taper off beyond 128, suggesting diminishing returns from additional context.

Although increasing the size of the fragment generally improves compression, the gain is not strictly linear. The most significant improvements are observed when transitioning from chunk size 16 to 64, whereas the relative improvement diminishes as chunk size increases beyond 128. This indicates that beyond a certain

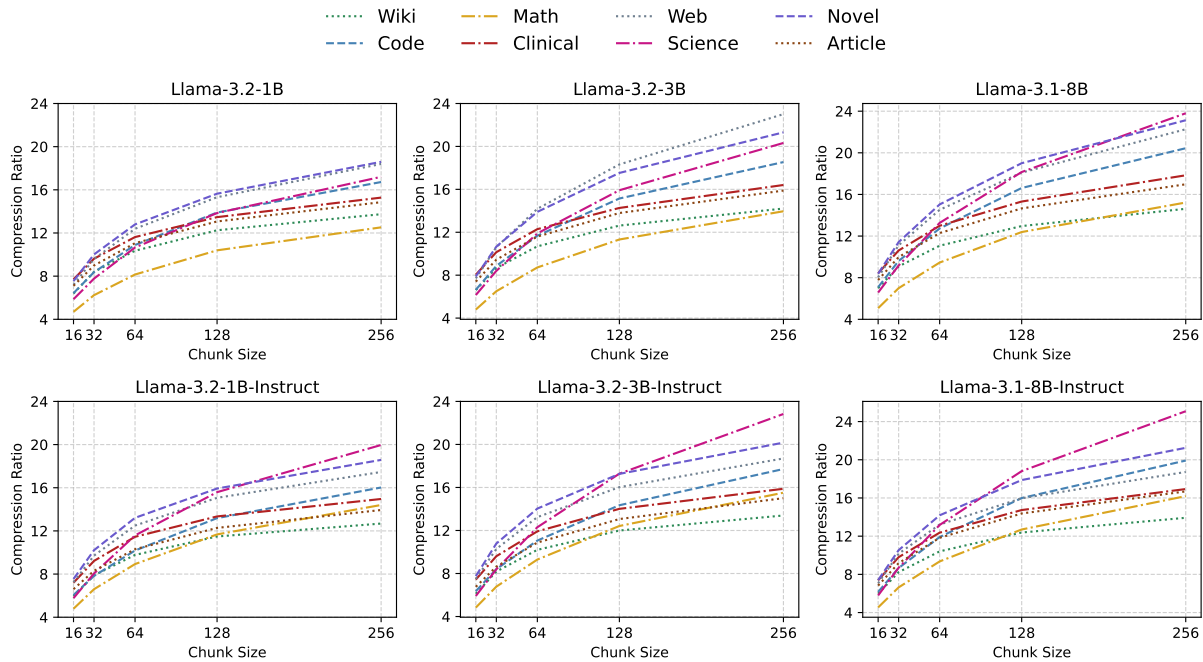


Figure 5: Compression ratio comparison across different Llama models. Larger models achieve higher compression ratios, demonstrating the advantage of increased model capacity in leveraging contextual information. Instruction-tuned variants generally perform slightly worse than their base counterparts on most datasets but achieve better compression on question-answering datasets.

threshold, the additional context provided by larger chunk sizes yields diminishing returns in terms of compression performance.

5.5 Impact of Model Size on Compression Performance

One of the most straightforward yet effective ways to improve compression efficiency is to increase the model size. In this subsection, we analyze the impact of model size on compression rate by examining Fig. 5, which illustrates that larger models consistently achieve higher compression rates across both base and instruction-tuned variants. To provide a more detailed comparison, we further include Fig. 6, which expands the analysis to six models: Apple1B, AMD1B, Llama1B, Llama3B, Llama8B, and Qwen14B.

The experimental results reveal a clear upward trend in compression ratio with increasing model size. Across all conditions, the smallest models exhibit the lowest compression rates, while the largest models, such as Llama8B and Qwen14B, demonstrate significantly higher compression performance. This pattern holds across different datasets and configurations, reinforcing the notion that larger models benefit from enhanced predictive capabilities, which allow them to reduce redundancy more effectively. Due to hardware constraints, our experiments are limited to models up to 14B parameters. We believe that 14B is sufficient to cover most LLM compression applications, as it already represents a large-scale model. However, we acknowledge that exploring even larger models remains an interesting direction for future research.

5.6 Impact of Dataset Scale on Compression Performance

Dataset scale is another crucial factor influencing compression efficiency. In this subsection, we investigate whether increasing the size of LLM-generated synthetic data affects compression performance. To this end, we evaluate both traditional and LLM-based compressors under different scales of the Wiki dataset.

The results are given in Fig. 7. Traditional methods such as Huffman coding, Arithmetic coding, FSE coding, and Gzip exhibit no significant change in compression rate, indicating that their underlying encoding mechanisms do not benefit from increased data availability. More advanced methods like LZMA and Zstd-22 show a slight improvement in compression rate with larger datasets, but the growth remains modest. In contrast, NN-based compressors such as TRACE and PAC demonstrate both superior compression performance and a slightly higher rate of improvement as dataset size increases. However, none of these methods come close to matching the compression efficiency achieved by the proposed approach.

A particular observation is that the proposed method maintains a consistent compression rate regardless of dataset scale. The stability of the proposed method across different dataset scales suggests potential applications in scenarios where data volume fluctuates, such as real-time streaming or incremental data storage.

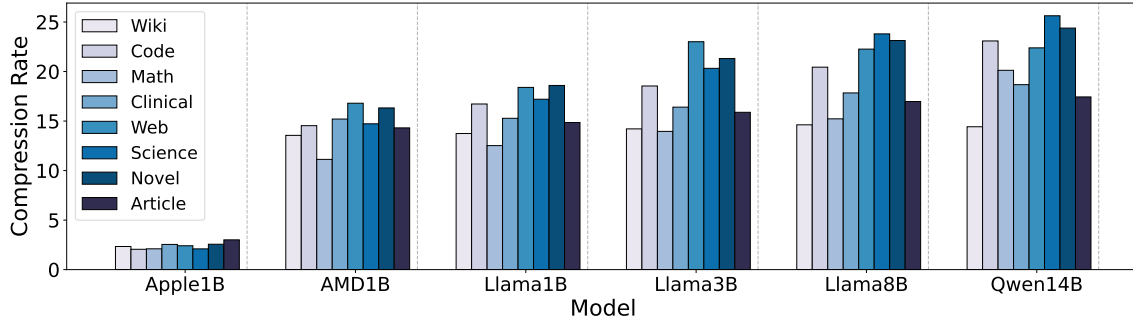


Figure 6: Model scale vs. compression ratio. Larger models consistently demonstrate higher compression ratios, indicating that increasing model size enhances predictive capabilities and further reduces redundancy.

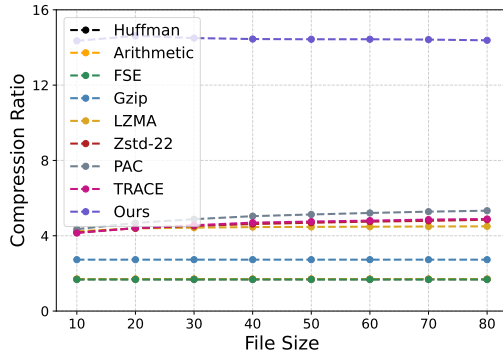


Figure 7: Compression ratio variations with increasing dataset size. Traditional methods remain stable, while neural compressors show slight improvement, but the proposed method consistently outperforms all approaches.

5.7 Comparison between LLMs with and without Fine-tuning

5.7.1 Instruction tuning. Instruction tuning is an indispensable post-training procedure for existing deployed LLMs, which aims to enable LLMs to align with human preference data and not exhibit unsafe behaviors. As instruction tuning also fine-tunes the token distributions of the base LLMs, in this subsection, we conduct an investigation into the influence of instruction tuning on the compression capabilities of LLMs for the text data.

The experimental results reveal that instruction-tuned variants generally exhibit a slightly lower compression ratio compared to their base model across most datasets. This phenomenon can be attributed to the differences in training objectives between pre-training and instruction tuning. During the initial training phase of large language models, the objective function is designed to minimize the cross-entropy loss between the predicted token distribution and the true labels, which aligns naturally with the goal of compression (minimizing entropy). However, instruction tuning introduces a different optimization objective that alters the token distribution, potentially reducing its effectiveness for compression.

Unlike pretraining, instruction tuning focuses on aligning model outputs with human preferences rather than purely maximizing predictive accuracy. This process involves reinforcement learning with human feedback (RLHF) or supervised fine-tuning on instruction-following datasets, where the model is explicitly optimized to generate responses that are coherent, contextually relevant, and stylistically appropriate rather than merely predicting the most probable next token. These factors can lead to a token distribution that is less compressible, as the model is encouraged to generate structured, human-like responses rather than the statistically optimal token sequences for entropy minimization.

One example supporting this hypothesis is the observation that instruction-tuned models achieve a consistent increase in compression efficiency on science and math datasets across different model sizes. Since these datasets primarily consist of structured question-answer pairs, their underlying probability distributions are more aligned with the instruction-tuning objective, which emphasizes coherent and structured responses. This alignment likely results in improved token predictability within these domains, leading to better compression rates.

5.7.2 Specific Domain Fine-tuning. To verify the compressibility of domain-specific models on their respective domains, we evaluate compression ratios using specialized models on math and code datasets. For the math dataset, we select Qwen2.5-Math-1.5B and Rho-Math-1B. For the code dataset, we use Qwen2.5-Coder-1.5B and DeepSeek-Coder-1.3B-Instruct. To ensure fairness, all selected models have sizes ranging from approximately 1B to 1.5B.

Table 8 presents the compression performance of general models and two domain-specific models on math and code tasks. While larger models such as Qwen14B and Llama8B generally achieve higher compression ratios, the domain-specific models exhibit significantly better compression efficiency. On the math dataset, when compared to similarly sized 1B models, Qwen2.5-Math-1.5B and Rho-Math-1B achieve noticeably higher compression ratios. Remarkably, Rho-Math-1B reaches a compression ratio comparable to Qwen14B, despite being 14 times smaller in size. For the code dataset, Qwen2.5-Coder-1.5B attains a compression ratio similar to Llama8B. However, DeepSeek-Coder-1.3B performs comparably to the general-purpose Llama1B model, which we suspect may be due to insufficient training. This suggests that models explicitly

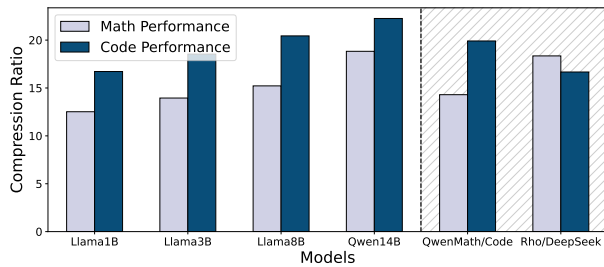


Figure 8: Performances of domain-specific models on Math and Code tasks. Domain-specific models consistently achieve higher compression ratios within their respective domains.

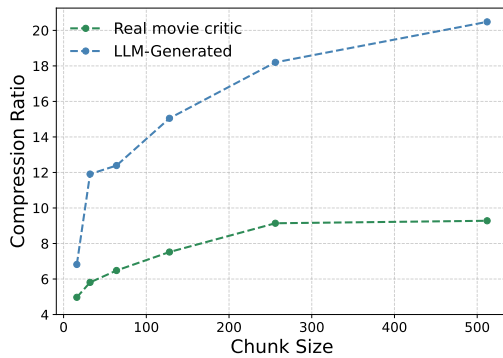


Figure 9: Compression ratio comparison between human-generated and LLM-generated text. LLMs achieve significantly higher compression ratios on LLM-generated data, with the gap widening as chunk size increases.

trained or fine-tuned for specific domains can better capture the structural patterns and redundancies inherent in domain-specific data, leading to improved compression rates.

5.8 Predictability of LLMs on Human vs. LLM-Generated Data

In this subsection, we study the predictability of LLMs for human-generated data and LLM-generated data. We evaluate the compression ratios of LLMs under different chunk sizes for the two categories of text data. Specifically, we run Llama-3.2-1B on real movie critics extracted from imdb and LLM-generated movie critics.

The results are shown in Figure 9. We can find that under different chunk sizes, LLMs demonstrate significantly higher compression ratios on LLM-generated data over human data. More interestingly, along with the increase in the chunk size, the compression ratios increase even further. Therefore, in large-scale data compression, as the ratios of LLM-generated data keep growing, LLM-based compressors will demonstrate increasingly significant compression capabilities compared to traditional compressors.

6 Related Work

6.0.1 Traditional Compression. Traditional lossless compression methods leverage entropy coding and dictionary-based techniques

to efficiently reduce data size while preserving exact information. Huffman coding [24] assigns variable-length codes based on symbol frequency, ensuring shorter codes for more common symbols. Arithmetic coding [51] provides better compression by representing entire sequences as fractional numbers, achieving near-optimal entropy efficiency. Dictionary-based methods like Gzip [15] and LZMA [58] utilize Lempel-Ziv compression, replacing repeated substrings with references to previous occurrences. Zstd [10], an advanced variant, improves upon Gzip with faster compression and decompression speeds while maintaining high compression ratios.

6.0.2 Neural Network-based Compressors. The use of neural networks as predictors for lossless compression has been widely explored, particularly in conjunction with arithmetic coding [3, 12, 19]. NNCP [3] employs transformers to estimate probability distributions, while TRACE [34] introduces a lightweight transformer variant to accelerate the compression process. Some works start using foundation models for lossless compression recently [14, 23, 40]. Studies leveraging models such as LLaMA[38] and Chinchilla [14] demonstrate promising results, particularly in compressing previously unseen modalities—for instance, large language models trained on text effectively compress images and audio. However, none of these studies specifically address *the domain of synthetic data*, leaving a gap in exploring how such models perform in artificial data compression scenarios.

7 Conclusion

As LLM-generated data continues to grow exponentially, effective lossless compression of LLM-generated data has become a critical challenge in text data management. In this work, we conducted the first systematic investigation into compressing LLM-generated text. We demonstrated that traditional lossless compressors struggle to effectively compress LLM-generated data due to the irregularity and complexity of LLM-generated data. Unlike previous machine-generated data, LLM-generated data lacks structured redundancy and appears more similar to human-generated data, posing critical challenges to lossless compression. To mitigate the issue, we proposed leveraging the predictive capabilities of LLMs originating from next-token prediction. Our extensive experiments with 14 LLMs and 8 datasets revealed that LLM-based prediction methods achieve over 20× compression, far surpassing Gzip’s 3× compression. Furthermore, we show that this approach generalizes across different model scales and dataset categories, underscoring the robustness and versatility of LLM-based compression.

References

- [1] Ayad Akhtar. 2024. The Role of AI in Creative Writing: A Case Study on McNeal. *The Atlantic* (2024). <https://www.theatlantic.com/magazine/archive/2024/11/mcneal-play-akhtar-downey-artificial-intelligence/679949/>.
- [2] Yushi Bai, Jiajie Zhang, Xin Lv, Linzhi Zheng, Siqi Zhu, Lei Hou, Yuxiao Dong, Jie Tang, and Juanzi Li. 2024. LongWriter: Unleashing 10,000+ Word Generation from Long Context LLMs. *arXiv preprint arXiv:2408.07055* (2024).
- [3] Fabrice Bellard. 2023. NNCP: Neural Network Compression Program. <https://bellard.org/nncp/>. Accessed: 2024-02-19.
- [4] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya

- Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *Advances in Neural Information Processing Systems*.
- [5] Randal E. Bryant, Randy H. Katz, and Edward D. Lazowska. 2008. Big-Data Computing: Creating Revolutionary Breakthroughs in Commerce, Science, and Society. (2008). https://cra.org/cra/wp-content/uploads/sites/2/2015/05/Big_Data.pdf Accessed: 2025-03-01.
 - [6] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott M. Lundberg, Harsha Nori, Hamid Palangi, Marco Túlio Ribeiro, and Yi Zhang. 2023. Sparks of Artificial General Intelligence: Early experiments with GPT-4. *arXiv preprint arXiv:2303.12712* (2023).
 - [7] D. Carrell and R. Smith. 2024. Legal AI: Applications and Risks of Large Language Models in Law. *Harvard Law Review* 137 (2024), 112–130. <https://www.reuters.com/technology/artificial-intelligence/ai-hallucinations-court-papers-spell-trouble-lawyers-2025-02-18/>.
 - [8] M. et al. Chen. 2021. Evaluating Large Language Models for Code Generation. *arXiv preprint arXiv:2107.03374* (2021). <https://arxiv.org/abs/2107.03374>.
 - [9] Yann Collet. 2013. Finite State Entropy. <https://github.com/Cyan4973/FiniteStateEntropy>. Accessed: 2025-03-01.
 - [10] Y. Collet. 2016. Zstd github repository from facebook. <https://github.com/facebook/zstd>
 - [11] Transaction Processing Performance Council. 2011. TPC-H. <http://www.tpc.org/tpch/default.asp>
 - [12] D. Cox. 2016. Syntactically informed text compression with recurrent neural networks. *arXiv preprint arXiv:1608.02893* (2016).
 - [13] DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Ruoyu Zhang, Runxin Xu, Qihao Zhu, Shiroong Ma, Peiyi Wang, Xiao Bi, Xiaokang Zhang, Xingkai Yu, Yu Wu, Z. F. Wu, Zhibin Gou, Zhihong Shao, Zhuoshu Li, Ziyi Gao, Aixin Liu, Bing Xue, Bingxuan Wang, Bochao Wu, Bei Feng, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, Damai Dai, Deli Chen, Dongjie Ji, Erhang Li, Fangyun Lin, Fucong Dai, Fuli Luo, Guangbo Hao, Guanting Chen, Guowei Li, H. Zhang, Han Bao, Hanwei Xu, Haocheng Wang, Honghui Ding, Huajian Xin, Huazuo Gao, Hui Qu, Hui Li, Jianzhong Guo, Jiashi Li, Jiawei Wang, Jingchang Chen, Jingyang Yuan, Junjie Qiu, Junlong Li, J. L. Cai, Jiaqi Ni, Jian Liang, Jin Chen, Kai Dong, Kai Hu, Kaige Gao, Kang Guan, Kexin Huang, Kuai Yu, Lean Wang, Lecong Zhang, Liang Zhao, Litong Wang, Liyue Zhang, Lei Xu, Leyi Xia, Mingchuan Zhang, Minghua Zhang, Minghui Tang, Meng Li, Miaojun Wang, Mingming Li, Ning Tian, Panpan Huang, Peng Zhang, Qiancheng Wang, Qinyu Chen, Qiusi Du, Ruiqi Ge, Ruisong Zhang, Ruizhe Pan, Runji Wang, R. J. Chen, R. L. Jin, Ruyi Chen, Shanghao Lu, Shangyan Zhou, Shanhuang Chen, Shengfeng Ye, Shiyu Wang, Shuiping Yu, Shunfeng Zhou, Shuting Pan, and S. S. Li. 2025. DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning. (2025). *arXiv:2501.12948*
 - [14] G. Delétang, A. Ruoss, P. Duquenne, E. Catt, T. Genewein, C. Mattern, J. Graumoya, L. K. Wenliang, M. Aitchison, L. Orseau, M. Hutter, and J. Veness. 2024. Language modeling is compression. In *International Conference on Learning Representations (ICLR)*.
 - [15] Peter Deutsch. [n. d.]. GZIP file format specification version 4.3. *RFC 1952* ([n. d.]).
 - [16] Dmitry Lepikhin et al. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *Proceedings of NeurIPS*. <https://arxiv.org/abs/2006.16668>
 - [17] Angela Fan, Mike Lewis, and Yann Dauphin. 2018. Hierarchical Neural Story Generation. In *Proceedings of ACL*. <https://arxiv.org/abs/1805.04833>
 - [18] Markus Freitag and Yaser Al-Onaizan. 2017. Beam search strategies for neural machine translation. *arXiv preprint arXiv:1702.01806* (2017).
 - [19] M. Goyal, K. Tatwawadi, S. Chandak, and I. Ochoa. 2019. Deepzip: Lossless data compression using recurrent neural networks. In *Data Compression Conference (DCC)*.
 - [20] Andrew Gray. 2023. ChatGPT "contamination": estimating the prevalence of LLMs in the scholarly literature. https://en.wikipedia.org/wiki/Generative_artificial_intelligence.
 - [21] Daya Guo, Qihao Zhu, Dejian Yang, Zhenda Xie, Kai Dong, Wentao Zhang, Guanting Chen, Xiao Bi, Y. Wu, Y.K. Li, Fuli Luo, Yingfei Xiong, and Wenfeng Liang. 2024. DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence. <https://arxiv.org/abs/2401.14196>
 - [22] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. 2020. The Curious Case of Neural Text Degeneration. In *Proceedings of ICLR*. <https://arxiv.org/abs/1904.09751>
 - [23] Y. Huang, J. Zhang, Z. Shan, and J. He. 2024. Compression represents intelligence linearly. *arXiv preprint arXiv:2404.09937* (2024).
 - [24] D. A. Huffman. 1952. *A Method for the Construction of Minimum-Redundancy Codes*. Vol. 40. 1098–1101 pages. doi:10.1109/JRPROC.1952.273884 Accessed: 2024-11-07.
 - [25] Binyuan Hui, Jian Yang, Zeyu Cui, Jiayi Yang, Dayiheng Liu, Lei Zhang, Tianyu Liu, Jiajun Zhang, Bowen Yu, Kai Dang, et al. 2024. Qwen2. 5-Coder Technical Report. *arXiv preprint arXiv:2409.12186* (2024).
 - [26] IBM Corporation. 2013. Bringing Big Data and Business Analytics to the Enterprise. (2013). https://public.dhe.ibm.com/software/os/systemz/pdf/Bringing_Big_Data_and_Business_Analytics_to_the_Enterprise.pdf Accessed: 2025-03-01.
 - [27] Robert A. Jacobs, Michael I. Jordan, Steven J. Nowlan, and Geoffrey E. Hinton. 1991. Adaptive Mixtures of Local Experts. *Neural Computation* 3, 1 (03 1991), 79–87.
 - [28] Prabha Kannan. 2023. How Much Research Is Being Written by Large Language Models? https://en.wikipedia.org/wiki/Generative_artificial_intelligence.
 - [29] Diederik P. Kingma and Jimmy Ba. 2015. Adam: A Method for Stochastic Optimization. In *International Conference on Learning Representations*.
 - [30] Sunjun Kwon, Junu Kim, Jiyoung Kim, Sujeong Im, Eunbyeol Cho, Seongsu Bae, Jungwoo Oh, Gyubok Lee, Jong Hak Moon, Seng Chan You, Seungjin Baek, Chang Hoon Han, Yoon Bin Jung, Yohan Jo, and Edward Choi. 2023. Publicly Shareable Clinical Large Language Model Built on Synthetic Clinical Notes. *arXiv:2309.00237* [cs.CL]
 - [31] Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for "Mind" Exploration of Large Scale Language Model Society. *arXiv:2303.17760* [cs.AI]
 - [32] Zhenghao Lin, Zhibin Gou, Yeyun Gong, Xiao Liu, Yelong Shen, Ruochen Xu, Chen Lin, Yujiu Yang, Jian Jiao, Nan Duan, and Weizhu Chen. 2024. Rho-1: Not All Tokens Are What You Need. *arXiv:2404.07965* [cs.CL]
 - [33] Jiang Liu, Jialian Wu, Prakamya Mishra, Zicheng Liu, Sudhanshu Ranjan, Pratik Prabhakaran Brahma, Yusheng Su, Gowtham Ramesh, Peng Sun, Zhe Li, Dong Li, Lu Tian, and Emad Barsoum. 2024. AMD-OLMo: A series of 1B language models trained from scratch by AMD on AMD Instinct™ MI250 GPUs. <https://huggingface.co/amd/AMD-OLMo>
 - [34] Yu Mao, Yufei Cui, Tei-Wei Kuo, and Chun Jason Xue. 2022. Trace: A fast transformer-based general-purpose lossless compressor. In *Proceedings of the ACM Web Conference 2022*. 1829–1838.
 - [35] Yu Mao, Jingzong Li, Yufei Cui, and Jason Chun Xue. 2023. Faster and stronger lossless compression with optimized autoregressive framework. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*. IEEE, 1–6.
 - [36] Sachin Mehta, Mohammad Hossein Sekhavat, Qingqing Cao, Maxwell Horton, Yanzi Jin, Chenfan Sun, Iman Mirzadeh, Mahyar Najibi, Dmitry Belenko, Peter Zatloukal, and Mohammad Rastegari. 2024. OpenELM: An Efficient Language Model Family with Open Training and Inference Framework. *arXiv.org* (April 2024). <https://arxiv.org/abs/2404.14619v1>
 - [37] Meta. 2024. Introducing Llama 3.1: Our most capable models to date. <https://ai.meta.com/blog/meta-llama-3-1/>.
 - [38] Meta. 2024. Llama 3.2: Revolutionizing edge AI and vision with open, customizable models. <https://ai.meta.com/blog/llama-3-2-connect-2024-vision-edge-mobile-devices/>.
 - [39] Arindam Mitra, Hamed Khanpour, Corby Rosset, and Ahmed Awadallah. 2024. Orca-Math: Unlocking the potential of SLMs in Grade School Math. *arXiv:2402.14830* [cs.CL]
 - [40] F. Mittu, Y. Bu, A. Gupta, A. Devireddy, A. E. Ozdarendeli, A. Singh, and G. Anumanchipalli. 2024. Finezip: Pushing the limits of large language models for practical lossless text compression. *arXiv preprint arXiv:2409.17141* (2024).
 - [41] Andreas Bentzen Winje Nicolai Thoror Sivesind. 2023. Human-vs-Machine.
 - [42] OpenAI. 2022. ChatGPT. <https://chat.openai.com/chat/>.
 - [43] OpenAI. 2023. ChatGPT. <https://en.wikipedia.org/wiki/ChatGPT>.
 - [44] OpenAI. 2023. GPT-4 Technical Report. *arXiv:2303.08774* [cs.CL]
 - [45] Claude Elwood Shannon. 1948. A mathematical theory of communication. *The Bell system technical journal* 27, 3 (1948), 379–423.
 - [46] Qwen Team. 2024. Qwen2.5: A Party of Foundation Models. <https://qwenlm.github.io/blog/qwen2.5/>
 - [47] Brian Thompson, Mehak Dhaliwal, Peter Frisch, Tobias Domhan, and Marcello Federico. 2024. A Shocking Amount of the Web is Machine Translated: Insights from Multi-Way Parallelism. https://en.wikipedia.org/wiki/Generative_artificial_intelligence.
 - [48] Hugo Touvron, Albert Jiang, et al. 2023. LLaMA: Open and Efficient Foundation Language Models. *arXiv preprint* (2023). <https://arxiv.org/abs/2302.13971>
 - [49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is All you Need. In *Advances in Neural Information Processing Systems*. 5998–6008.
 - [50] Y. Wang and J. Lee. 2023. Applications of Large Language Models in Medical Diagnostics and Education. *Journal of Medical AI Research* 45 (2023), 23–35. <https://pmc.ncbi.nlm.nih.gov/articles/PMC11396764/>.
 - [51] Witten et al. 1987. Arithmetic coding for data compression. *Commun. ACM* 30, 6 (1987), 520–540.
 - [52] Jinzhao Xiao, Yuxiang Huang, Changyu Hu, Shaoxu Song, Xiangdong Huang, and Jianmin Wang. 2022. Time series data encoding for efficient storage: a comparative analysis in Apache IoTDB. *Proc. VLDB Endow.* 15, 10 (June 2022), 2148–2160. doi:10.14778/3547305.3547319
 - [53] Li Xu, Yanqing He, Guanghui Li, Wei Li, and Wen Shi. 2021. Big Data-Driven Smart Manufacturing: Challenges, Advances and Future Trends. *IEEE Transactions on Industrial Informatics* 17, 3 (2021), 2274–2284.

- [54] Vishaal Y. 2025. Synthetic Code Generations Dataset. <https://huggingface.co/datasets/VishaalY/synthetic-code-generations>.
- [55] An Yang, Beichen Zhang, Binyuan Hui, Bofei Gao, Bowen Yu, Chengpeng Li, Dayiheng Liu, Jianhong Tu, Jingren Zhou, Junyang Lin, Keming Lu, Mingfeng Xue, Runji Lin, Tianyu Liu, Xingzhang Ren, and Zhenru Zhang. 2024. Qwen2.5-Math Technical Report: Toward Mathematical Expert Model via Self-Improvement. *arXiv preprint arXiv:2409.12122* (2024).
- [56] Hailin Zhang, Penghao Zhao, Xupeng Miao, Yingxia Shao, Zirui Liu, Tong Yang, and Bin Cui. 2023. Experimental Analysis of Large-Scale Learnable Vector Storage Compression. *Proc. VLDB Endow.* 17, 4 (Dec. 2023), 808–822. doi:10.14778/3636218.3636234
- [57] Lixi Zhou, Jiaqing Chen, Amitabh Das, Hong Min, Lei Yu, Ming Zhao, and Jia Zou. 2022. Serving deep learning models with deduplication from relational databases. 15, 10 (June 2022), 2230–2243. doi:10.14778/3547305.3547325
- [58] Ziv et al. 1977. A universal algorithm for sequential data compression. *IEEE Transactions on information theory* 23, 3 (1977), 337–343.